

Automated Task Reminder and Tracking Application

Internship Project Documentation

1. Introduction to the Project

The **Automated Task Reminder and Tracking Application** is a backend system developed using **Spring Boot**. The purpose of this application is to allow users to create, manage, and track tasks while receiving reminders before deadlines.

The system stores task information in a database and automatically manages reminders using scheduling features provided by Spring Boot.

This project helps demonstrate the use of:

- Spring Boot framework
- REST API development
- Database interaction using Spring Data JPA
- Model-View-Controller architecture
- Task scheduling features

Objectives of the Project

The main objectives of the project are:

- Create and manage tasks
- Store task information in a database
- Track task completion status
- Send reminders before deadlines
- Organize application logic using Spring Boot architecture

2. Introduction to Spring Framework

The **Spring Framework** is one of the most popular frameworks used for developing enterprise-level Java applications.

Spring provides a comprehensive infrastructure that helps developers build robust, scalable, and maintainable applications.

It reduces the complexity of Java development by providing built-in solutions for many common problems such as dependency management, database access, security, and web development.

Key Features of Spring Framework

1. Dependency Injection (DI)

Dependency Injection is a design pattern used by Spring to manage object dependencies automatically. Instead of creating objects manually, Spring creates and injects them where required.

2. Inversion of Control (IoC)

In traditional programming, the developer controls object creation. In Spring, the **Spring Container** controls the creation and lifecycle of objects.

3. Aspect-Oriented Programming (AOP)

AOP allows developers to separate cross-cutting concerns such as logging, security, and transactions from the main business logic.

4. Integration Support

Spring integrates easily with various technologies such as:

- Hibernate
- JDBC
- REST APIs
- Microservices
- Cloud services

3. Introduction to Spring Boot

Spring Boot is an extension of the Spring Framework that simplifies the process of building Java applications.

Before Spring Boot, developers had to configure many files manually. Spring Boot eliminates this complexity by providing **auto configuration** and **embedded servers**.

This allows developers to build and run applications quickly without complicated setup.

Key Features of Spring Boot

1. Auto Configuration

Spring Boot automatically configures the application based on the dependencies added in the project.

For example, if Spring Web dependency is added, Spring Boot automatically configures REST controllers.

2. Embedded Server

Spring Boot applications include an embedded server such as:

- Tomcat
- Jetty
- Undertow

This means the application can run directly without deploying it separately on a server.

3. Standalone Application

Spring Boot applications can run independently using a simple **main method**.

4. Dependency Management

Spring Boot uses **Maven or Gradle** to manage project dependencies automatically.

4. Architecture of the Application

The Automated Task Reminder and Tracking Application follows a layered architecture commonly used in Spring Boot applications.

The main layers include:

Controller Layer

Handles incoming HTTP requests from users and returns responses.

Service Layer

Contains business logic and application rules.

Repository Layer

Responsible for communication with the database.

Model Layer

Represents the structure of the data stored in the database.

This separation improves the **maintainability and scalability** of the application.

5. Steps to Create the Spring Boot Project

The following steps were followed to create the project.

Step 1 – Open IDE

The project was created using **IntelliJ IDEA**.

Step 2 – Create New Project

Select **Spring Initializr** while creating the new project.

Step 3 – Configure Project Details

Fill the following information:

Group:

com.example

Artifact:

taskreminder

Project Name:

Automated Task Reminder and Tracking Application

Language:

Java

Build Tool:

Maven

Step 4 – Add Dependencies

The following dependencies were added to the project:

- Spring Web
 - Spring Data JPA
 - MySQL Driver
 - Lombok
-

Step 5 – Generate Project

After adding dependencies, the project structure is generated automatically.

6. Spring Boot Application Entry Point

Every Spring Boot application starts from a main class containing the **@SpringBootApplication** annotation.

Code Example

```
@SpringBootApplication
public class TaskReminderApplication {

    public static void main(String[] args) {
        SpringApplication.run(TaskReminderApplication.class, args);
    }
}
```

Explanation

@SpringBootApplication

This annotation combines three annotations:

- @Configuration

- `@EnableAutoConfiguration`
- `@ComponentScan`

It enables Spring Boot features and starts the application.

7. Model Class in the Project

The **Model class** represents the structure of the data stored in the database.

In the Automated Task Reminder and Tracking Application, the model class is **TaskModel**.

Each object of the model represents a task stored in the database.

Important Fields in the Model

id

Unique identifier for each task.

title

Represents the name of the task.

description

Provides detailed information about the task.

dueDate

Represents the deadline of the task.

status

Indicates whether the task is pending or completed.

Model Class Code Example

```
package com.example.taskreminder.model;
```

```
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.Id;
```

```
import java.time.LocalDateTime;
```

```
@Entity
```

```
public class TaskModel {
```

```
@Id
private int id;

private String title;
private String description;
private LocalDateTime dueDate;
private String status;
}
```

Explanation

@Entity

Marks the class as a database entity.

@Id

Defines the primary key of the table.

LocalDateTime

Used to store date and time information for task deadlines.

8. Variables and Data Types Used

Variables are used to store data in a program. Java provides several data types.

Common Data Types Used in the Project

Data Type	Description
int	Used to store numeric values
String	Used to store text
boolean	Used to store true or false values
LocalDateTime	Used to store date and time

Example:

```
int id = 1;  
String title = "Complete Assignment";  
boolean completed = false;
```

9. Java Collection Framework

The **Java Collection Framework (JCF)** provides classes and interfaces used to store and manipulate groups of objects.

Collections are used in applications to manage large amounts of data efficiently.

Advantages of Collection Framework

- Dynamic memory allocation
 - Built-in data structures
 - Efficient searching and sorting
 - Reusable code
-

Main Interfaces in Java Collection Framework

List Interface

A **List** stores ordered elements and allows duplicates.

Examples:

- ArrayList
- LinkedList

Example:

```
List<String> tasks = new ArrayList<>();  
tasks.add("Submit Assignment");  
tasks.add("Attend Meeting");
```

Set Interface

A **Set** stores unique elements and does not allow duplicates.

Examples:

- HashSet
- TreeSet

Example:

```
Set<Integer> taskIds = new HashSet<>();  
taskIds.add(1);  
taskIds.add(2);
```

Map Interface

A **Map** stores data in key-value pairs.

Examples:

- HashMap
- TreeMap

Example:

```
Map<Integer, String> taskMap = new HashMap<>();  
taskMap.put(1, "Complete report");  
taskMap.put(2, "Prepare presentation");
```

10. Spring Boot Annotations Used in the Project

Annotations are special markers used to provide metadata to the compiler and Spring framework.

They help simplify configuration and reduce the need for XML configuration files.

Important Annotations Used

@SpringBootApplication

Used to start the Spring Boot application.

@RestController

Indicates that the class is a REST controller used to handle HTTP requests.

@RequestMapping

Maps web requests to specific controller methods.

@GetMapping

Handles HTTP GET requests.

@PostMapping

Handles HTTP POST requests.

@Autowired

Automatically injects dependencies into classes.

@Service

Marks a class as a service component containing business logic.

@Repository

Indicates that the class interacts with the database.

11. Conclusion

The **Automated Task Reminder and Tracking Application** demonstrates the use of the Spring Boot framework for developing backend applications.

Through this project, the following concepts were implemented:

- Spring Framework fundamentals
- Spring Boot architecture

- REST API development
- Database interaction using Spring Data JPA
- Model class design
- Java Collection Framework
- Spring Boot annotations

This project provides a strong foundation for developing scalable backend systems using Java and Spring Boot.

12. Spring Boot Annotations

Spring Boot provides a large number of annotations that help simplify the development of Java applications. Annotations are special markers used to provide metadata to the Spring Framework so that it can automatically configure the application.

Using annotations reduces the need for complex XML configuration files and allows developers to build applications quickly and efficiently.

In the **Automated Task Reminder and Tracking Application**, several Spring Boot annotations are used to manage application components such as controllers, services, repositories, and configuration classes.

Below are some important Spring Boot annotations used in the project.

@SpringBootApplication

The @SpringBootApplication annotation is the **main annotation used to start a Spring Boot application**. It is placed on the main class of the project.

This annotation is actually a combination of three annotations:

- @Configuration
- @EnableAutoConfiguration
- @ComponentScan

Code Example

```
@SpringBootApplication
public class TaskReminderApplication {

    public static void main(String[] args) {
        SpringApplication.run(TaskReminderApplication.class, args);
    }
}
```

Explanation

- **@Configuration**
Indicates that the class can be used by Spring as a source of bean definitions.
 - **@EnableAutoConfiguration**
Automatically configures the application based on the dependencies added in the project.
 - **@ComponentScan**
Scans the project packages to detect components such as controllers, services, and repositories.
-

@Component

The @Component annotation marks a class as a **Spring-managed component**. When Spring starts the application, it automatically detects this class and creates an object of it.

Example

```
@Component
public class TaskUtility {

}
```

Explanation

Spring will automatically create and manage the object of this class inside the **Spring Container**.

@Service

The @Service annotation is used to define **service layer classes**. These classes contain the **business logic** of the application.

Example

```
@Service
public class TaskService {

}
```

Explanation

The service layer processes data, applies rules, and communicates with the repository layer.

@Repository

The @Repository annotation marks classes that interact with the **database layer**.

Repository classes perform database operations such as:

- Saving data
- Fetching data
- Updating data
- Deleting data

Example

```
@Repository
public interface TaskRepository extends JpaRepository<TaskModel, Integer> {

}
```

Explanation

Spring Data JPA automatically provides built-in methods for database operations.

@Autowired

The @Autowired annotation is used for **dependency injection**. It allows Spring to automatically inject objects where they are required.

Example

```
@Autowired
private TaskService taskService;
```

Explanation

Instead of creating objects manually using the new keyword, Spring automatically provides the required object.

@Configuration

The @Configuration annotation indicates that the class contains configuration methods for the application.

Example

```
@Configuration
public class AppConfig {

}
```

Explanation

Configuration classes define beans and application settings.

@Bean

The @Bean annotation is used to define objects that should be managed by the Spring container.

Example

```
@Bean
public TaskService taskService() {
    return new TaskService();
}
```

Explanation

Spring will create and manage the lifecycle of this object.

13. REST API in Spring Boot

REST stands for **Representational State Transfer**. It is an architectural style used for building web services that allow communication between different applications over HTTP.

In the **Automated Task Reminder and Tracking Application**, REST APIs are used to perform operations such as:

- Creating tasks
- Updating tasks
- Retrieving tasks
- Deleting tasks
- Tracking task status

Spring Boot provides several annotations that simplify the creation of REST APIs.

@RestController

The `@RestController` annotation is used to create REST API controllers.

It combines two annotations:

- `@Controller`
- `@ResponseBody`

Example

```
@RestController
public class TaskController {

}
```

Explanation

This annotation tells Spring that the class will handle HTTP requests and return responses in **JSON format**.

@RequestMapping

The `@RequestMapping` annotation maps HTTP requests to specific controller classes or methods.

Example

```
@RequestMapping("/tasks")
```

Explanation

This defines the base URL for all task-related APIs.

Example endpoints:

```
/tasks/create
```

```
/tasks/update
```

```
/tasks/delete
```

@GetMapping

The @GetMapping annotation is used to retrieve data from the server.

Example

```
@GetMapping("/tasks")
public List<TaskModel> getAllTasks() {
    return taskService.getAllTasks();
}
```

Explanation

This method retrieves all tasks stored in the database.

@PostMapping

The @PostMapping annotation is used to create new data in the database.

Example

```
@PostMapping("/tasks")
public TaskModel createTask(@RequestBody TaskModel task) {
    return taskService.saveTask(task);
}
```

Explanation

- Receives task data from the client.

- Saves the task in the database.
-

@PutMapping

The @PutMapping annotation is used to update existing records.

Example

```
@PutMapping("/tasks/{id}")
public TaskModel updateTask(@PathVariable int id, @RequestBody TaskModel task) {
    return taskService.updateTask(id, task);
}
```

Explanation

Updates the task with the specified ID.

@DeleteMapping

The @DeleteMapping annotation is used to delete data from the database.

Example

```
@DeleteMapping("/tasks/{id}")
public String deleteTask(@PathVariable int id) {
    taskService.deleteTask(id);
    return "Task deleted successfully";
}
```

Explanation

Deletes the task with the given ID from the database.

@RequestBody

The @RequestBody annotation is used to bind HTTP request data to a Java object.

Example

```
@PostMapping("/tasks")
public TaskModel createTask(@RequestBody TaskModel task) {
```

```
}
```

Explanation

The JSON request body is automatically converted into a Java object.

@PathVariable

The @PathVariable annotation extracts values from the URL.

Example

```
@GetMapping("/tasks/{id}")
public TaskModel getTask(@PathVariable int id) {

}
```

Explanation

The ID from the URL is passed to the method.

Example request:

GET /tasks/1

14. Frontend Development

In this project, a simple frontend is developed using **HTML, CSS, and Thymeleaf**.

Technologies Used

- HTML → Structure of pages
 - CSS → Styling and UI design
 - Thymeleaf → Dynamic data rendering
-

Pages Created

1. Login Page (login.html)

- Allows user to enter email and password
- Redirects to dashboard

2. Register Page (register.html)

- Allows user to create account
- Takes basic user details

3. Dashboard Page (dashboard.html)

- Displays all tasks
 - Shows task status and due dates
-

CSS Styling

A separate CSS file is used:

src/main/resources/static/css/style.css

Features of CSS:

- Centered login/register form
 - Modern UI with spacing and colors
 - Styled buttons and inputs
 - Responsive layout
-

Connecting CSS

CSS is linked using:

```
<link rel="stylesheet" href="/css/style.css">
```

15. Task Reminder Scheduler

The application includes an **automatic reminder system** using Spring Boot Scheduler.

Scheduler Purpose

- Runs automatically every 1 minute
- Checks all tasks

- Prints reminder if task is overdue
-

Code Concept

```
@Scheduled(fixedRate = 60000)
public void checkTasks() {
    List<TaskModel> tasks = taskManager.getTasks();

    for (TaskModel task : tasks) {
        if (task.getStatus() != null &&
            task.getStatus().equalsIgnoreCase("pending") &&
            task.getDueDate().isBefore(LocalDate.now())) {

            System.out.println("Reminder: Task is overdue!");
        }
    }
}
```

Annotation Used

@EnableScheduling

 Enables scheduling in Spring Boot

16. Application Flow

 Step-by-step working of your project:

1. User opens application → /login
2. User logs in → redirected to dashboard
3. Dashboard requests data from controller
4. Controller calls Service layer
5. Service calls Repository
6. Repository fetches data from database

7. Data is sent back to frontend
8. Scheduler runs in background
9. If task overdue → reminder printed

17. Database Integration

The project uses **MySQL database**.

Configuration (application.properties)

```
spring.datasource.url=jdbc:mysql://localhost:3306/taskdb
spring.datasource.username=root
spring.datasource.password=your_password
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Table Structure

Field	Type
id	int
title	varchar
description	varchar
dueDate	datetime
status	varchar

18. GitHub Integration

The project is uploaded to GitHub using Git commands.

Steps Followed

```
git init
git add .
git commit -m "Initial commit"
git remote add origin <repo-link>
git pull origin main --allow-unrelated-histories
git push -u origin main
```

License Used

MIT License is added to allow open usage of the project.

19. Challenges Faced

During development, the following challenges were faced:

- Git merge conflict issues
 - Vim editor blocking commit process
 - CSS not loading due to wrong static path
 - Database column mismatch (status not found)
 - NullPointerException in scheduler
-

Solutions

- Used Notepad instead of Vim
- Fixed static folder structure
- Added status column in database
- Added null check in scheduler